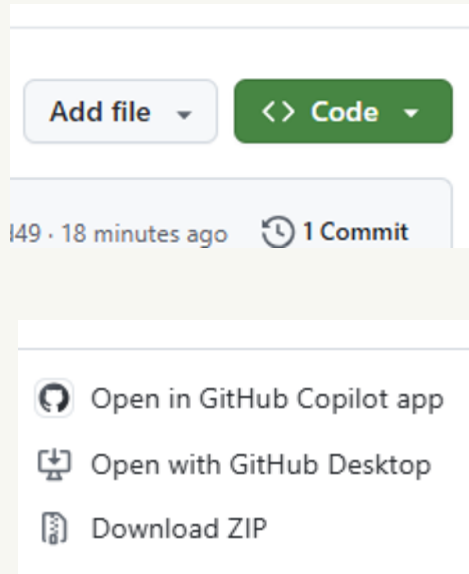


https://github.com/it-nci/Python_workshop

1. Click Code
2. Download ZIP



WORKSHOP · 5 ต.ค. 2569

Python for Data Analysis @NCI

เวิร์กชอปเรียนรู้การวิเคราะห์ข้อมูล ด้วย Python



Basic python · Data analysis · Data visualization

Mindset

1. Python เป็น tool หนึ่ง (ที่ทดแทน excel ได้แบบไร้ขีดจำกัด) ไม่ใช่เฉพาะการเขียนโปรแกรม
2. Domain knowledge และ กระบวนการคิดเป็นขั้นเป็นตอน สำคัญว่า syntax หรือ ไวยากรณ์ของ
ภาษาคอมพิวเตอร์
3. Error จากการ run code เป็นเรื่องปกติ และไม่ทำร้ายใคร ดังนั้น การฝึกฝน การทดลอง บ่อยๆ จะทำให้เก่งขึ้น
4. ฝึกทำกับงานจริง สำคัญกว่าการท่องจำ

Data Engineer

Up to 40,000 THB
(ตามประสบการณ์)



ลาดปลาเค้า 25 BTSเสนานิคม

รายละเอียดงาน

- พัฒนาและปรับปรุง Customer Data Platform (CDP) บน Google Cloud Platform (GCP)
- ดูแลและแก้ไขปัญหา Data Pipeline ด้วย Apache Airflow, SQL และเครื่องมือบน GCP
- ใช้ Python (pandas) วิเคราะห์และจัดการข้อมูล เพื่อสนับสนุนงานของแผนกต่าง ๆ
- ตรวจสอบและรับประกันคุณภาพข้อมูล ให้ถูกต้อง ครบถ้วน และสม่ำเสมอ
- ทำงานร่วมกับทีมวิศวกรข้อมูล แลกเปลี่ยนความรู้ และช่วยเหลือทีมงานรุ่นใหม่

คุณสมบัติ

- วุฒิการศึกษาปริญญาตรี สาขาวิทยาการคอมพิวเตอร์ วิศวกรรมศาสตร์
- มีประสบการณ์ทำงานด้าน Data Engineering อย่างน้อย 3 ปีขึ้นไป
- มีความรู้และทักษะในการใช้ Apache Airflow สำหรับสร้างและจัดการ Data Workflow
- มีทักษะเชี่ยวชาญในการใช้ SQL และการจัดการฐานข้อมูล
- มีประสบการณ์ใช้ Python (โดยเฉพาะ pandas) เพื่อจัดการและวิเคราะห์ข้อมูล
- มีประสบการณ์ใช้เครื่องมือ GCP data tools (โดยเฉพาะ Dataform หรือ SQLX)



สมัครงานติดต่อ คุณพลอย 063 904 7237

<<< Scan เพื่อส่ง CV ได้เลยนะคะ

AGENDA

Workshop นี้มี 3 ช่วง

1

Basic python

ตัวแปร และชนิดข้อมูล

2

Data analysis

จัดการและเตรียมข้อมูลตาราง ด้วย Pandas

3

Data visualization

สร้างกราฟ ด้วย Seaborn และ Plotly

ลักษณะเด่นของ Python

เรียนรู้ได้ไม่ยาก

ระดับความยากอยู่ที่ย่างถึงปานกลาง

รันได้ที่ละบรรทัด

ง่ายต่อการเรียนรู้ ทดสอบ และตรวจสอบผล

Open-source และฟรี

มีผู้พัฒนา library ต่างๆ ทั่วโลก

ใช้ได้ทุก platform

รวมถึงรันบนเว็บ เช่น Google Colab ที่ใช้อยู่

PYTHON

Python ใช้ทำอะไรได้

เป็นที่นิยม

วิเคราะห์ข้อมูล และทำ **Data visualization**

เป็นที่นิยม

Data science, Machine learning และ **Deep learning**

พัฒนาเว็บไซต์

Automation และ **scripting**

พัฒนา **software / program**

JUPYTER / GOOGLE COLAB (.IPYNB)

ข้อดีของ **Python Notebook** ในการวิเคราะห์ข้อมูล

Code + Text + กราฟ

ผสมไว้ในที่เดียว รันดูผลทีละเซลล์ได้ทันที

เหมาะกับการสำรวจข้อมูล

ทำ data cleansing, transformation และทดลองปรับโมเดลได้สะดวก

แสดงผลได้ทันที

กราฟจาก Matplotlib, Seaborn, Plotly และตาราง Pandas

แบ่งปันง่าย

ผู้อื่นนำไฟล์ .ipynb ไปรันซ้ำและตรวจสอบผลได้

เหมาะกับงานเตรียมข้อมูล

ทำงานที่ละชั้น และตรวจสอบได้

แก้เฉพาะจุด

ไม่ต้องสร้างใหม่ตั้งแต่ต้นทุกครั้ง
ประหยัดเวลากับข้อมูลขนาดใหญ่

ทำซ้ำได้

รันซ้ำได้เมื่อมีข้อมูลชุดใหม่เข้ามา

ตรวจสอบได้

อธิบายขั้นตอนและตรวจสอบข้อมูลที่ละชั้นตอน
ได้

GOOGLE COLAB

การใช้งาน Google Colab

File · Edit · View · Runtime · Tools · Sidebar

File: จัดการไฟล์ Notebook

คำสั่ง	ใช้ทำอะไร
New notebook	สร้างสมุดงานใหม่ เก็บที่ Drive (โฟลเดอร์ Colab Notebooks) อัตโนมัติ
Open notebook	เปิดไฟล์จาก Drive, GitHub หรืออัปโหลดจากเครื่อง
Save a copy in Drive	สำคัญมาก: ก๊อปปี้ไว้ในบัญชีตัวเอง ก่อนแก้ notebook ของคนอื่น
Save a copy in GitHub	push ขึ้น repo ได้ตรงๆ (authorize ครั้งแรกครั้งเดียว)
Revision history	ดูประวัติการแก้ไข ย้อนกลับเวอร์ชันเก่าได้
Download	โหลดเป็น .ipynb (เปิดใน Jupyter ได้) หรือ .py (สคริปต์ Python ล้วน)

Edit: แก้ไขเซลล์และเนื้อหา

คำสั่ง	ใช้ทำอะไร
Undo / Redo	ย้อนการกระทำ
Select all cells	เลือกทุกเซลล์พร้อมกัน
Cut / Copy / Paste cell	ตัด คัดลอก วางทั้งเซลล์
Delete selected cells	ลบเซลล์ที่เลือก
Find and replace	ค้นหาและแทนที่ข้อความทั้ง notebook
Clear all outputs	ล้างผลลัพธ์ทั้งหมด ไฟล์เล็กลงมาก่อนส่งต่อหรือ push ขึ้น GitHub
Notebook settings	ตั้งค่า runtime type

View และ Tools

View: ปรับมุมมอง

Table of contents

เปิดสารบัญด้านซ้าย เดินทางระหว่างหัวข้อได้เร็ว

Executed code history

ดูโค้ดที่เคยรันในเซลล์ ช่วยตอนเพลออลบเซลล์

Collapse / Expand sections

ยุบหรือขยายทุกหัวข้อ

Show / hide code

ซ่อนโค้ด เหลือแต่ผลลัพธ์ เหมาะตอนนำเสนอ

Tools: ที่สำคัญ

Command palette (Ctrl + Shift + P)

พิมพ์ชื่อคำสั่งแล้วเรียกใช้ทันที ไม่ต้องไล่หาในเมนู

Settings

ธีมสีเข้ม (Dark mode), ขนาดฟอนต์, การเติมโค้ดอัตโนมัติ

Keyboard shortcuts

ดูและแก้ไขคีย์ลัดทั้งหมด

Runtime: ควบคุมการรันโค้ด

คำสั่ง	ใช้ทำอะไร
Run all	รันทุกเซลล์ตั้งแต่บนลงล่าง
Interrupt execution	หยุดโค้ดที่รันค้าง เช่น <code>loop</code> ไม่รู้จบ
Restart session	รีสตาร์ท <code>Python</code> ใหม่ ตัวแปรหายแต่ไฟล์ยังอยู่ ใช้แก้ปัญหาไลบรารีชนกันหลัง <code>pip install</code>
Restart and run all	รีสตาร์ทแล้วรันใหม่ทั้งหมด วิธีเช็กที่ดีที่สุดว่า <code>notebook</code> ทำงานได้จริงตั้งแต่ต้น
Disconnect and delete runtime	ระวัง: ลบเครื่องทิ้ง ไฟล์ทั้งหมดใน <code>/content</code> จะหายด้วย

Runtime: คำสั่งอื่นที่ควรรู้

Run before / Run after

รันเฉพาะเซลล์ก่อนหน้า หรือตั้งแต่เซลล์ปัจจุบันลงไป

Change runtime type

เลือก CPU / GPU / TPU (ดูหน้าถัดไป)

View resources

ดูการใช้ RAM, Disk, GPU แบบเรียลไทม์

Manage sessions

ดูว่า notebook ไหนกินเครื่องอยู่ ปิดตัวที่ไม่ใช้เพื่อคืนทรัพยากร

เปิดใช้ GPU / TPU

1

ไปที่เมนู Runtime

2

เลือก Change runtime
type

3

ในช่อง Hardware
accelerator เลือก T4 GPU

4

กด Save

ข้อควรระวัง: การเปลี่ยน runtime type จะรีสตาร์ทเซสชัน ตัวแปรทั้งหมดที่รันไว้จะหาย ควรเปลี่ยนก่อนเริ่มทำงาน

แถบเครื่องมือด้านซ้ายและด้านล่าง

Table of contents · สารบัญหัวข้อ

Find and replace · ค้นหา / แทนที่

Code snippets · โค้ดสำเร็จรูป

Data inspector · ดูหน้าตาข้อมูลในตัวแปร

Secrets · เก็บ API key อย่างปลอดภัย

Files · อัปโหลด / ดาวน์โหลด / เชื่อม Drive

Data explorer · โหลด public dataset มาเล่น

Variables · ดูตัวแปรทั้งหมดในหน่วยความจำ

ด้านล่าง

Terminal · ส่งคำสั่งควบคุมระบบ Linux โดยตรง

Toggle Gemini · เรียก Gemini มาช่วย

NEXT

Session 1

Basic python fundamentals

ทดสอบการใช้งานโปรแกรมครั้งแรก

```
print("Hello World!")
```

Hello World!

1.1 ตัวแปรและชนิดของข้อมูล

ตัวแปร (Variable)

คือกล่องติดป้ายชื่อ

ตั้งชื่อให้กล่อง แล้วใส่ข้อมูลลงไปข้างใน

`age = 45`

`age` = ป้ายชื่อของกล่อง

`45` = ข้อมูลที่ใส่ในกล่อง

1.1 ตัวแปรและชนิดของข้อมูล

ทำไมเราถึงต้องใช้ตัวแปร?

แทนที่จะเขียนแบบนี้

พิมพ์ 45 ซ้ำ ๆ ทุกที่

ต้องไล่แก้ทีละจุดเมื่อข้อมูลเปลี่ยน

ให้เขียนแบบนี้

ใช้ **age** (อายุ) แทน

โค้ดอ่านเข้าใจง่ายขึ้น และแก้ไขข้อมูลภายหลังได้สะดวกขึ้น

1.1 ตัวแปรและชนิดของข้อมูล

ชนิดข้อมูลพื้นฐานใน Python

ชนิดข้อมูล (Type)	ตัวอย่าง	ใช้กับ
string (str)	"John Doe"	ข้อความ ชื่อ รหัสต่างๆ
integer	45	จำนวนเต็ม การนับจำนวน
float	36.5	ทศนิยม ค่าจากการวัด
boolean	True / False	จริง/เท็จ ใช่/ไม่ใช่ เปิด/ปิด

1.1 ตัวแปรและชนิดของข้อมูล

เหมือนกรอกแบบฟอร์มข้อมูลคนไข้

ชื่อบุคคล (ข้อความ)

string

```
patient_name = "John Doe"
```

อายุ (ตัวเลข)

integer

```
age = 45
```

อุณหภูมิร่างกาย (ทศนิยม)

float

```
temperature = 36.5
```

สูบบุหรี่หรือไม่? (ใช่/ไม่ใช่)

boolean

```
is_smoker = True
```

1.1 การแปลงชนิดข้อมูล

ปัญหาชนิดข้อมูลที่พบบ่อย

1

อายุเป็นข้อความ

"45"

นำไปคำนวณทางสถิติหรือหาค่าเฉลี่ยต่อไม่ได้

2

ระยะโรคมีจุดทศนิยม

2.0

จริงๆ ควรเป็นจำนวนเต็ม 2

3

เลขบัตรประชาชน (CID) เป็นตัวเลข

1234567890123

ควรเป็นข้อความ เพื่อป้องกันเลขศูนย์ตัวหน้าหาย
หรือฟอร์แมตเพี้ยน

1.1 การแปลงชนิดข้อมูล

ฟังก์ชันแปลงชนิดข้อมูล

ฟังก์ชัน	แปลงเป็น	ตัวอย่าง	ผลลัพธ์
int()	จำนวนเต็ม (Integer)	int("45")	45
float()	ทศนิยม (Decimal)	float("45.5")	45.5
str()	ข้อความ (String)	str(45)	"45"
bool()	จริง/เท็จ (True/False)	bool(1)	True

ตรวจสอบชนิดข้อมูลด้วย `type()` เช่น `type(age)` จะบอกว่า `age` เป็นชนิดใด

1.2 ตัวดำเนินการคำนวณ

Basic arithmetic operators

Operator	ความหมาย	ตัวอย่าง	ผลลัพธ์
+	บวก (Addition)	5 + 3	8
-	ลบ (Subtraction)	5 - 3	2
*	คูณ (Multiplication)	5 * 3	15
/	หาร (Division)	10 / 2	5.0
//	หารเอาจำนวนเต็ม (Floor Division)	10 // 3	3
%	เศษจากการหาร (Remainder)	10 % 3	1
**	ยกกำลัง (Power)	2 ** 3	8

1.2 ตัวดำเนินการคำนวณ

ข้อสังเกตของ / และ //

/

ผลลัพธ์เป็นทศนิยม (Decimal) เสมอ

$10 / 2 \rightarrow 5.0$

//

ผลลัพธ์เป็นจำนวนเต็ม (Whole number)

$10 // 3 \rightarrow 3$

%

เศษที่เหลือจากการหาร

$10 \% 3 \rightarrow 1$

1.3 ตัวแปรในการคำนวณ

ทำไมต้องใช้ตัวแปรในการคำนวณ?

แทนที่จะพิมพ์ตัวเลขเดิมซ้ำๆ เราเก็บตัวเลขไว้ในตัวแปรแทน ซึ่งช่วยให้โค้ด

อ่านและเข้าใจได้ง่ายขึ้น

Easier to read

แก้ไขหรืออัปเดตข้อมูลได้ง่ายขึ้น

Easier to update

ลดโอกาสเกิดข้อผิดพลาดได้มากกว่า

Less error-prone

1.3 ตัวแปรในการคำนวณ

ตัวอย่าง: คำนวณดัชนีมวลกาย (BMI)

แทนที่จะเขียนแบบนี้

```
print(70 / (1.75 ** 2))
```

ให้ใช้ตัวแปรแทน

```
weight_kg = 70  
height_m = 1.75  
bmi = weight_kg / (height_m ** 2)  
# Clear and readable!
```

1.4 การเปรียบเทียบ (COMPARISONS)

การเปรียบเทียบ คือการตั้งคำถาม

ตรวจสอบว่าเงื่อนไขเป็น จริง (True) หรือ เท็จ (False) เช่น

ผู้ป่วยรายนี้อายุมากกว่า **60** ปีหรือไม่?

ระยะของโรคเท่ากับ 'IV' หรือไม่?

วันที่วินิจฉัยโรคเกิดขึ้นหลังปี **2020**
หรือไม่?

1.4 การเปรียบเทียบ (COMPARISONS)

ตัวดำเนินการเปรียบเทียบ

Operator	ความหมาย	ตัวอย่าง
<code>==</code>	เท่ากับ	<code>age == 65</code>
<code>!=</code>	ไม่เท่ากับ	<code>sex != 'M'</code>
<code>></code>	มากกว่า	<code>age > 60</code>
<code><</code>	น้อยกว่า	<code>age < 50</code>
<code>>=</code>	มากกว่าหรือเท่ากับ	<code>age >= 60</code>
<code><=</code>	น้อยกว่าหรือเท่ากับ	<code>age <= 50</code>

1.4 ข้อผิดพลาดที่พบบ่อย

ระวัง: = กับ == ไม่เหมือนกัน

= กำหนดค่า (setting a value)

age = 45

กำหนดให้ age มีค่าเท่ากับ 45

== เปรียบเทียบ (checking equality)

age == 45

age มีค่าเท่ากับ 45 ใช่หรือไม่? (ตอบ True / False)

1.4 การเปรียบเทียบ (COMPARISONS)

การรวมหลายเงื่อนไข (Logical operators)

and

ต้องเป็นจริงทั้งหมด

```
age > 60 and stage == 'IV'
```

ทั้งอายุมากกว่า 60 และระยะโรคเป็น IV จึงจะเป็นจริง

or

เป็นจริงอย่างน้อยหนึ่งอย่าง

```
age > 60 or stage == 'IV'
```

อย่างใดอย่างหนึ่งเป็นจริง (หรือจริงทั้งคู่) ก็ถือว่าจริง

not

กลับค่าผลลัพธ์

```
not is_smoker
```

เป็นจริงถ้า is_smoker เป็น False (ไม่ได้สูบบุหรี่)

1.4 คำสั่ง IF

ตัดสินใจตามเงื่อนไข

if เงื่อนไข_1:

 ทำสิ่งนี้

elif เงื่อนไข_2:

 ทำสิ่งนี้แทน

else:

 ทำสิ่งนี้เมื่อไม่ตรงกับเงื่อนไขใดๆ เลย

ผังงาน (Flowchart)



1.4 การเปรียบเทียบ

คุณจะได้ใช้การเปรียบเทียบเพื่อ

กรองข้อมูล (Filter data)

แสดงเฉพาะผู้ป่วยสูงอายุ

ค้นหาข้อผิดพลาด (Find errors)

หาข้อมูลที่ระบุอายุมากกว่า 120 ปี

จัดกลุ่มข้อมูล (Categorize)

ถ้าเป็นระยะ I หรือ II ให้ระบุว่าเป็น 'ระยะเริ่มต้น (Early)'

ตรวจสอบความถูกต้อง (Validate)

ถ้าวันที่วินิจฉัยลงบันทึกเป็นวันในอนาคต ให้แจ้งเตือนข้อผิดพลาด

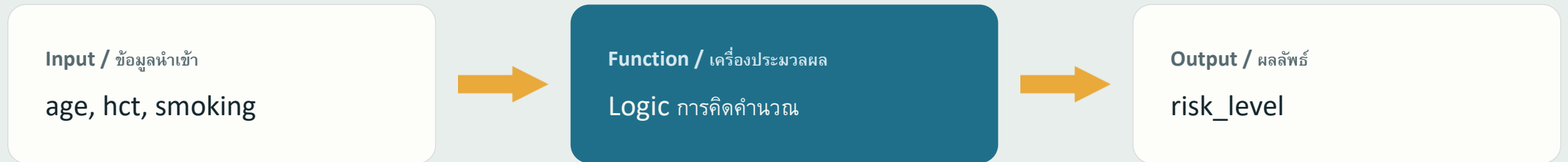
1.4 ตัวอย่างการใช้งานจริง

การตัดสินใจทางคลินิก (Clinical decision)

```
if age >= 60 and stage == 'IV':  
    print("ความเสี่ยงสูง - จำเป็นต้องติดตามอาการอย่างใกล้ชิด")  
elif age >= 60:  
    print("ความเสี่ยงปานกลาง - ตรวจติดตามอาการตามระยะ")  
else:  
    print("แนวทางการดูแลรักษามาตรฐาน")
```

1.5 FUNCTION

Function: ข้อมูลเข้า → ประมวลผล → ผลลัพธ์



ตัวอย่าง: ประเมินระดับความเสี่ยงของผู้ป่วยจากข้อมูลที่ส่งเข้าไป

1.5 FUNCTION

ทำไมเราต้องใช้ **Function**?

Reusability

ไม่ต้องเขียนโค้ดซ้ำ

เขียน **Logic** การตัดสินใจแค่ครั้งเดียว แต่เรียกใช้งานซ้ำกี่ครั้งก็ได้

Readability

โค้ดอ่านง่ายและเป็นระเบียบ

แทนที่จะเห็น `if-elif-else` ยาวๆ หลายสิบบรรทัด เรารวบแล้วตั้งชื่อให้ เช่น `assess_risk()`

1.5 FUNCTION

โครงสร้างของ Function ใน Python

สร้างฟังก์ชันด้วยคำสั่ง **def** (ย่อมาจาก Define)

```
def ชื่อฟังก์ชัน(ตัวแปรรับค่า1, ตัวแปรรับค่า2):  
    # โค้ดประมวลผลภายใน (ต้องย่อหน้าเข้ามา 4 ช่อง)  
    ...  
    return ผลลัพธ์ที่ต้องการส่งออก
```

1.5 FUNCTION

ตัวอย่าง: ประเมินความเสี่ยงผู้ป่วย

1. นิยามฟังก์ชัน

```
def assess_patient_risk(age, smoking):
```

```
    if age >= 65 and smoking:
```

```
        return "High Risk"
```

```
    elif age >= 65 or smoking:
```

```
        return "Moderate Risk"
```

```
    else:
```

```
        return "Low Risk"
```

2. เรียกใช้งานฟังก์ชัน (บรรทัดเดียวจบ)

```
assess_patient_risk(68, True) # High Risk
```

```
assess_patient_risk(45, False) # Low Risk
```

SESSION 2

Data analysis

Tabular Data Processing: จัดการและเตรียมข้อมูลตาราง ด้วย Pandas

2.1 อ่านไฟล์และดูข้อมูล

Library (ไลบรารี) คืออะไร?

ชุดเครื่องมือที่มีผู้พัฒนาเขียนไว้ให้แล้ว เพื่อเพิ่มความสามารถให้ Python เหมือนแอปที่ดาวโหลดมาติดตั้งเพิ่มบนมือถือ

Pandas

จัดการข้อมูลตาราง

ใช้กับข้อมูลที่อยู่ในรูปแบบตาราง คล้ายการใช้งาน Excel

NumPy

จัดการตัวเลข

ใช้กับตัวเลขและการคำนวณทางคณิตศาสตร์

2.1 อ่านไฟล์และดูข้อมูล

การเรียกใช้งาน (import)

```
import pandas as pd # โหลด pandas และตั้งชื่อเรียกแทนว่า pd (พิมพ์สั้นลง)
```

```
import numpy as np # โหลด numpy และตั้งชื่อเรียกแทนว่า np
```

import แต่ละไลบรารีแค่ครั้งเดียวก็พอ โดยมักเขียนไว้ที่บรรทัดแรกๆ ของโค้ด

2.1 อ่านไฟล์และดูข้อมูล

DataFrame คืออะไร?

ข้อมูลในรูปแบบตารางใน Python คล้ายแผ่นตารางงานของ Excel

rows

แถว

ข้อมูลของผู้ป่วยแต่ละคน / แต่ละเรคคอร์ด

columns

คอลัมน์

ฟิลด์ข้อมูล เช่น ชื่อ อายุ ผลการวินิจฉัยโรค

df

ชื่อตัวแปร

มักตั้งชื่อว่า df ตามธรรมเนียมของนักเขียนโปรแกรม

2.1 อ่านไฟล์และดูข้อมูล

การอ่านไฟล์ (Reading files)

ไฟล์ Excel

```
df = pd.read_excel('filename.xlsx', sheet_name='xxx')
```

ไฟล์ CSV

```
df = pd.read_csv('filename.csv')
```

ไฟล์ SPSS

```
df = pd.read_spss('filename.sav')
```

ไฟล์ SPSS ต้องติดตั้ง pyreadstat ก่อน

กี่แถว?

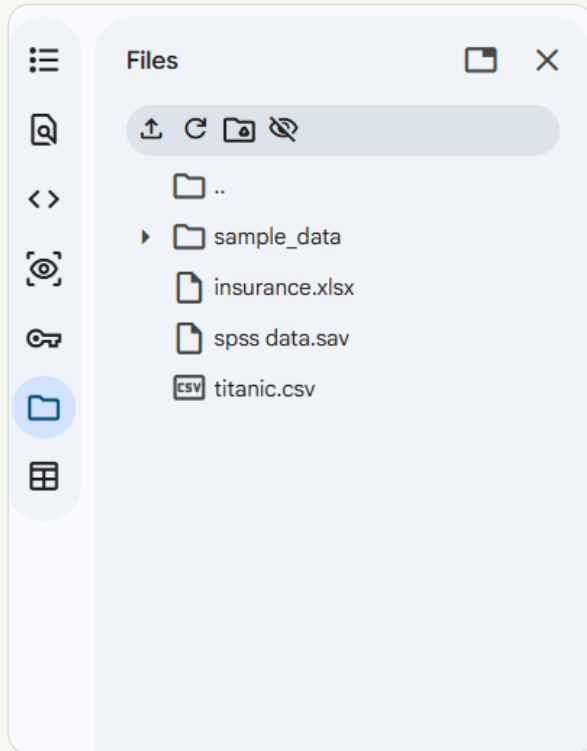
```
len(df) # มีผู้ป่วยกี่คน
```

กี่คอลัมน์?

```
len(df.columns) # มีฟิลด์ข้อมูลกี่หัวข้อ
```

2.1 อ่านไฟล์และดูข้อมูล

นำไฟล์จากเครื่องเข้า Colab



ไฟล์ที่นำเข้าจะอยู่ใน **folder /content/**

ข้อควรระวัง: ไฟล์ใน folder นี้อยู่ชั่วคราว ถ้าปิด Colab / ตัดการเชื่อมต่อ / restart จะหายไปทั้งหมด

วิธีป้องกัน

1. Link Colab กับ Google Drive
2. Download ลงเครื่องก่อนปิดหรือ restart session

2.1 ดูข้อมูลก่อนเริ่มทำความสะอาด

ก่อนเริ่ม ต้องตรวจสอบก่อนว่า

หน้าตาของข้อมูลเป็นอย่างไร?

มีคอลัมน์อะไรอยู่บ้าง?

มีความผิดปกติหรือปัญหาที่เห็นได้ชัดหรือไม่?

Best practice: เริ่มทำความสะอาดข้อมูลด้วยการรัน `df.head()`, `df.info()` และ `df.describe()` เสมอ เพื่อทำความเข้าใจโครงสร้างข้อมูลก่อนลงมือทำอย่างอื่น

2.1 ดูข้อมูลก่อนเริ่มทำความสะอาด

คำสั่งที่จำเป็นสำหรับการดูข้อมูล

คำสั่ง	สิ่งที่แสดงผล
df.head(n)	ข้อมูล n แถวแรก (ค่าเริ่มต้น 5 แถว)
df.tail(n)	ข้อมูล n แถวสุดท้าย (ค่าเริ่มต้น 5 แถว)
df.columns	รายชื่อคอลัมน์ทั้งหมด
df.info()	ชนิดข้อมูล และจำนวนข้อมูลที่ขาดหาย
df.describe()	สถิติเบื้องต้นของข้อมูลตัวเลข
df.shape	ขนาดข้อมูล (จำนวนแถว, จำนวนคอลัมน์)

2.2 เข้าถึงและกรองข้อมูล

เข้าถึงคอลัมน์ (Column)

เลือกคอลัมน์เดียว

```
df['ชื่อคอลัมน์']
```

แนะนำ ผลลัพธ์เป็น **Series**

```
df.ชื่อคอลัมน์
```

Dot notation ใช้ได้เฉพาะชื่อที่ไม่มีเว้นวรรคหรือสัญลักษณ์พิเศษ

เลือกหลายคอลัมน์

```
df[['Col1', 'Col2']]
```

ใส่ List ของชื่อคอลัมน์ซ้อนเข้าไป (วงเล็บ 2 ชั้น) ผลลัพธ์เป็น **DataFrame**

2.2 เข้าถึงและกรองข้อมูล

เข้าถึงแถว (Row): .loc และ .iloc

เครื่องมือ	หลักการทำงาน	ตัวอย่าง
.loc[]	อ้างอิงด้วย Label (ชื่อ Index)	'id_1', 'id_2'
.iloc[]	อ้างอิงด้วย Integer Position (ลำดับตัวเลข)	0, 1, 2 (เริ่มนับจาก 0)

ตำแหน่ง (iloc)	Index (loc)	Name	Age	City	Score
0	id_1	Alice	25	Bangkok	85.5
1	id_2	Bob	30	Chiang Mai	92.0
2	id_3	Charlie	35	Phuket	78.5
3	id_4	David	40	Pattaya	88.0

2.2 เข้าถึงและกรองข้อมูล

เข้าถึงและแก้ไขข้อมูลเฉพาะจุด (Cell)

อ้างอิงด้วยชื่อ

```
df.loc[Row_Label, Column_Name]
```

อ้างอิงด้วยลำดับตัวเลข

```
df.iloc[Row_Position, Column_Position]
```

แก้ไขค่าในเซลล์เดียวด้วย **.at**

เข้าถึงและแก้ไขข้อมูลในเซลล์เดียว (Scalar Value) ได้มีประสิทธิภาพและเร็วที่สุดใน Pandas ส่วน **.iat** ใช้ลำดับตำแหน่งแทนชื่อ

```
df.at[Row_Label, Column_Name] = ค่าใหม่
```

2.2 เข้าถึงและกรองข้อมูล

การเลือกข้อมูลแบบมีเงื่อนไข

Boolean Indexing: ใช้เงื่อนไขกรองเอาเฉพาะแถวที่ต้องการ โดยมากใช้ AND (&), OR (|) และ .isin()

&

แทน **AND** (และ)

(a) & (b)

ต้องผ่านทุกเงื่อนไข

|

แทน **OR** (หรือ)

(a) | (b)

ผ่านเงื่อนไขใดเงื่อนไขหนึ่งก็พอ

.isin()

เช็คค่าในกลุ่มข้อมูล

df['City'].isin([...])

ค่าในคอลัมน์ตรงกับค่าใดค่าหนึ่งใน List ที่กำหนดหรือไม่

2.2 ข้อควรระวัง

ต้องใส่วงเล็บครอบแต่ละเงื่อนไข

ห้ามใช้คำว่า and / or ใน pandas ให้ใช้ & และ | แทน

ผิด: เกิด **Error**

```
df[df['Age'] > 25 & df['Score'] > 80]
```

ถูกต้อง

```
df[(df['Age'] > 25) & (df['Score'] > 80)]
```

2.2 เข้าถึงและกรองข้อมูล

เปรียบเทียบ: ใช้ | กับ .isin()

ต้องการหาคนที่อยู่ในเมือง Bangkok, Phuket หรือ Pattaya

แบบเดิม: ใช้ | ต่อกันหลายตัว (ยาว อ่านยาก ต้องใส่วงเล็บหลายชั้น)

```
df[(df['City'] == 'Bangkok') | (df['City'] == 'Phuket') | (df['City'] == 'Pattaya')]
```

ใช้ .isin() (กระชับ อ่านง่ายขึ้น)

```
df[df['City'].isin(['Bangkok', 'Phuket', 'Pattaya'])]
```

ใช้ร่วมกับ & (อยู่ใน Bangkok หรือ Phuket และ Score มากกว่า 80)

```
df[(df['City'].isin(['Bangkok', 'Phuket'])) & (df['Score'] > 80)]
```

2.4 ค่าว่าง (MISSING DATA)

ทำไมต้องตรวจข้อมูลที่ขาดหาย?

ข้อมูลที่หายไป (Missing Data) จะส่งผลกระทบต่อ

ความถูกต้องแม่นยำของการวิเคราะห์

Analysis accuracy

ความสมบูรณ์ของรายงาน

Report completeness

การคำนวณทางสถิติ

Statistical calculations

2.4 ค่าว่าง (MISSING DATA)

"ค่าว่าง" คืออะไร?

ใน Pandas ข้อมูลที่ขาดหายไปจะแสดงผลในรูปแบบเหล่านี้

NaN

ย่อมาจาก Not a Number

None

ค่าว่างแบบหนึ่งของ Python

ช่องว่างเปล่า

Empty cells ที่ติดมาจากการโหลดไฟล์ Excel / CSV

2.4 ค่าว่าง (MISSING DATA)

การค้นหาข้อมูลที่ขาดหาย

คำสั่ง	ผลลัพธ์
<code>df.isnull()</code>	True / False ทุกช่อง (True = ค่าว่าง)
<code>df.isnull().sum()</code>	จำนวนค่าว่างในแต่ละคอลัมน์ (ใช้บ่อยมาก!)
<code>df.isnull().sum().sum()</code>	จำนวนค่าว่างทั้งหมดในตาราง
<code>df[df['stage'].isnull()]</code>	เฉพาะแถวที่คอลัมน์ stage ขาดหาย
<code>df[df.isnull().any(axis=1)]</code>	เฉพาะแถวที่มีค่าว่างในคอลัมน์ใดก็ได้

2.4 ค่าว่าง (MISSING DATA)

กลยุทธ์การจัดการข้อมูลที่ขาดหาย

1 ลบทิ้ง (Remove)

```
df.dropna()
```

2 เติมด้วยค่าคงที่

```
df.fillna(0)
```

เช่น เติมด้วยเลข 0

3 เติมด้วยค่าเฉลี่ย

```
df['age'].fillna(df['age'].mean())
```

เช่น เอาอายุเฉลี่ยมาเติมให้คนที่อายุหายไป

4 กลับไปตรวจสอบที่มา

ตรวจว่าทำไมข้อมูลนี้ถึงหายไปตั้งแต่แรก

2.4 ค่าว่าง (MISSING DATA)

ตัวอย่างการจัดการข้อมูล (Titanic)

Age หายไป

```
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

Cabin หายไป

```
df['Cabin'] = df['Cabin'].fillna('Unknown')
```

Ticket หายไป

```
df = df.dropna(subset=['Ticket'])
```

แทนด้วยค่าเฉลี่ย · เติมคำว่า Unknown · ลบแถวทิ้ง

2.5 ข้อมูลซ้ำ (DUPLICATES)

การตรวจข้อมูลซ้ำ (.duplicated)

คืนค่า True / False โดย **True** = แถวนั้นซ้ำกับแถวก่อนหน้า

```
df[df.duplicated(subset=['xxx', 'yyy'], keep='first')]
```

พารามิเตอร์	ความหมาย
subset= ['Col1', 'Col2']	คอลัมน์ที่ใช้เช็คซ้ำ (ถ้าไม่ใส่ = เช็คทุกคอลัมน์)
keep= 'first' (Default)	เว้นแถวแรกไว้ แถวซ้ำถัดมาเป็น True
keep= 'last'	เว้นแถวสุดท้ายไว้ แถวซ้ำก่อนหน้าเป็น True
keep= False	Mark เป็น True ทุกแถวที่มีคู่ซ้ำ

2.5 ข้อมูลซ้ำ (DUPLICATES)

ตัวอย่างผลของ .duplicated()

Name	Age	City	Score	ซ้ำทุกคอลัมน์?	ซ้ำที่ Name?
Alice	25	Bangkok	85	False	False
Bob	30	Chiang Mai	90	False	False
Alice	25	Bangkok	85	True	True
Charlie	35	Phuket	78	False	False
Bob	30	Pattaya	92	False	True

ซ้ำทุกคอลัมน์: 1 แถว (Alice แถวที่ 3) · ซ้ำเฉพาะ Name: 2 แถว (Alice และ Bob) ซ้ำหรือไม่ขึ้นกับคอลัมน์ที่เลือก

2.5 ข้อมูลซ้ำ (DUPLICATES)

การลบข้อมูลซ้ำ (.drop_duplicates)

```
df.drop_duplicates(subset=['xxx', 'yyy'], keep='first', inplace=False)
```

Parameter	ค่าที่ใช้ได้	คำอธิบาย
subset	['Col1', 'Col2']	คอลัมน์ที่ใช้เช็คซ้ำ (ถ้าไม่ใส่ = ทุกคอลัมน์)
keep	'first', 'last', False	เลือกตัวที่เก็บไว้ (False = ไม่เก็บเลย)
inplace	True, False	True = แก้ไขลงใน DataFrame เดิมทันที

2.5 ข้อมูลซ้ำ (DUPLICATES)

เก็บแถวที่มีค่ามากที่สุดไว้

Pandas ไม่มี keep='max' หรือ keep='min' จึงใช้ 2 ขั้น

1

เรียงข้อมูล (**.sort_values**)

ให้ค่าที่ต้องการเก็บอยู่อันดับแรก

2

ลบข้อมูลซ้ำ (**.drop_duplicates**)

ใช้ keep='first' (หรือ 'last')

```
df.sort_values('Score', ascending=False).drop_duplicates(subset=['Name'], keep='first')
```

2.7 DATA STANDARDIZATION

ค่าเดียวกัน แต่เขียนหลายแบบ

ข้อมูลในคอลัมน์มักหลากหลายทั้งที่มีความหมายเดียวกัน เช่น เพศชายอาจเป็น M, Male, 1 ต้องแปลงให้เป็นมาตรฐานเดียวกัน

ค่าที่พบ	แปลงเป็น	ความหมาย
m, MALE, MAN, 1	'1'	เพศชาย
f, Female, WOMAN, 2	'2'	เพศหญิง
ค่าอื่นๆ เช่น Boy, Unknown	'error'	เตือนให้กลับไปตรวจสอบข้อมูล

2.7 DATA STANDARDIZATION

ใช้ Function ร่วมกับเงื่อนไข if

```
def standardize_sex(sex_str):  
    sex_str = sex_str.upper() # แปลงเป็นพิมพ์ใหญ่ก่อน  
    if sex_str in ['1', 'M', 'MALE', 'MAN']:  
        return '1'  
    elif sex_str in ['2', 'F', 'FEMALE', 'WOMAN']:  
        return '2'  
    else:  
        return 'error'  
  
df['Sex_Cleaned'] = df['Sex_Raw'].apply(standardize_sex)
```

2.7 DATA STANDARDIZATION

ผลลัพธ์: Sex_Raw → Sex_Cleaned

m → 1	Female → 2
MALE → 1	2 → 2
MAN → 1	f → 2
Boy → error	WOMAN → 2
Unknown → error	1 → 1

แถวที่ได้ **'error'** (Boy, Unknown) คือค่าที่ยังไม่อยู่ในรายการ ให้กลับไปตรวจสอบข้อมูลหรือเพิ่มคำศัพท์ใหม่เข้าไปในเงื่อนไข

2.8 สรุปข้อมูลตามกลุ่ม (GROUPBY)

groupby: แบ่งกลุ่ม → คำนวณ → รวมผล

ตอบคำถามที่พบบ่อยที่สุด เช่น ผู้ป่วยแต่ละจังหวัดมีกี่ราย ทำงานเหมือน Pivot Table ใน Excel

1 · split

แบ่งกลุ่ม

แยกแถวตามค่าในคอลัมน์ เช่น First /
Second / Third

2 · apply

คำนวณ

นับ / รวม / เฉลี่ย ภายในแต่ละกลุ่ม

3 · combine

รวมผล

ได้ตารางสรุป 1 แถวต่อ 1 กลุ่ม

```
df.groupby("คอลัมน์ที่ใช้แบ่งกลุ่ม")["คอลัมน์ที่จะคำนวณ"].วิธีคำนวณ()
```

2.8 สรุปข้อมูลตามกลุ่ม (GROUPBY)

วิธีคำนวณที่ใช้บ่อย

คำสั่ง	ได้อะไรในแต่ละกลุ่ม
.size()	จำนวนแถว (นับทุกแถว รวมแถวที่มีค่าว่าง)
.count()	จำนวนค่าที่ไม่ว่างในคอลัมน์นั้น
.sum()	ผลรวม
.mean() / .median()	ค่าเฉลี่ย / ค่ามัธยฐาน
.min() / .max()	ค่าต่ำสุด / สูงสุด
.agg(...)	หลายค่าพร้อมกันในตารางเดียว

เคล็ดลับ: คอลัมน์ 0/1 เช่น survived → .sum() = จำนวนคนรอด และ .mean() = อัตรารอด

2.8 สรุปข้อมูลตามกลุ่ม (GROUPBY)

ตัวอย่าง: แบ่งกลุ่มด้วย 1 คอลัมน์ และ 2 คอลัมน์

อัตราการรอดชีวิต แยกตามชั้นตัว

```
(df_clean.groupby("class")["survived"]  
    .mean())
```

class	survived
First	0.63
Second	0.47
Third	0.24

จำนวนคน แยกตามชั้นตัว X เพศ

```
(df_clean.groupby(["class", "sex"])  
    .size().unstack())
```

class	female	male
First	92	122
Second	76	108
Third	144	347

unstack() พลิกกลุ่มที่ 2 (sex) ขึ้นไปเป็นคอลัมน์ ได้ตารางแบบ Pivot Table อ่านง่ายขึ้น

2.8 สรุปข้อมูลตามกลุ่ม (GROUPBY)

หลายค่าสรุปพร้อมกันด้วย .agg()

```
summary = df_clean.groupby("class").agg(  
    n_passengers=("survived", "size"),    # จำนวนคน  
    n_survived=("survived", "sum"),        # จำนวนคนรอด  
    survival_rate=("survived", "mean"),    # อัตรารอด  
    mean_age=("age", "mean"),              # อายุเฉลี่ย  
) .round(2).reset_index()                # reset_index: ให้ class เป็นคอลัมน์ปกติ
```

class	n_passengers	n_survived	survival_rate	mean_age
First	214	134	0.63	36.69
Second	184	87	0.47	29.77
Third	491	119	0.24	25.93

2.9 เชื่อม 2 ตาราง (MERGE)

ประเภทการเชื่อมตาราง (Join)

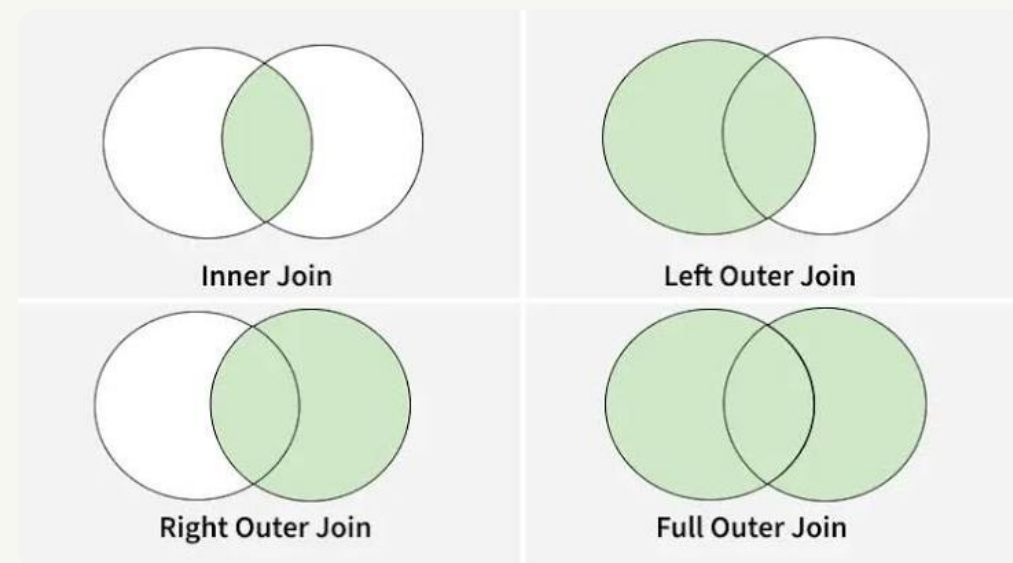
นำข้อมูลจาก 2 ตารางมาต่อกันข้างๆ โดยใช้ "รหัสอ้างอิงร่วม"

Inner Join ดึงเฉพาะข้อมูลที่มีตรงกันทั้ง 2 ตาราง

Left Join ข้อมูลตารางซ้ายมาครบ + ตารางขวาเฉพาะที่จับคู่ได้

Right Join ข้อมูลตารางขวามาครบ + ตารางซ้ายเฉพาะที่จับคู่ได้

Full Join รวมข้อมูลทั้งหมดของทั้ง 2 ตารางเข้าด้วยกัน



2.9 เชื่อม 2 ตาราง (MERGE)

pd.merge()

```
pd.merge(df1, df2, on='ID', how='outer', indicator=True)
```

พารามิเตอร์	ความหมาย
on	คอลัมน์หรือสัณฐานที่ใช้อ้างอิงร่วมกัน
how	ประเภทการเชื่อม: 'inner', 'left', 'right', 'outer'
indicator	True = เพิ่มคอลัมน์ <code>_merge</code> บอกว่าแต่ละแถวมาจากตารางไหน
suffixes	ต่อท้ายชื่อคอลัมน์ที่ซ้ำกัน เช่น ('_web', '_event')

2.9 เชื่อม 2 ตาราง (MERGE)

ตัวอย่าง: คำสั่งซื้อ + ข้อมูลลูกค้า

df_orders (3 คำสั่งซื้อ) เชื่อมกับ df_users (3 คน) ด้วย user_id โดยยึดตารางคำสั่งซื้อเป็นหลัก

```
pd.merge(df_orders, df_users, on='user_id', how='left')
```

order_id	user_id	price	name
101	1	500	Somchai
102	2	1200	Somsri
103	1	300	Somchai

2.9 เชื่อม 2 ตาราง (MERGE)

ใช้ `merge` ตรวจสอบข้อมูลซ้ำระหว่าง 2 ตาราง

```
pd.merge(members_web, members_event, on='name', how='outer', indicator=True)
```

both

ซ้ำกัน (มีในทั้ง 2 ชุด)

บอบ, แคท: 2 คน

left_only

มีเฉพาะชุดแรก (เว็บ)

แอน, แดน, อีฟ: 3 คน

right_only

มีเฉพาะชุดที่สอง (Event)

แฟรงค์, กัส, กัส: 3 แถว

สังเกต: กัส ปรากฏ 2 แถวเพราะซ้ำอยู่ใน `members_event` เอง ควรตรวจสอบและลบแถวซ้ำภายในตารางเดียวกันก่อน `merge`

2.10 ต่อตาราง (CONCAT)

pd.concat(): ต่อ DataFrame หลายตัว

axis=0 (Default)

ต่อแนวตั้ง

เอาแถวมาต่อลงมาเรื่อยๆ (เพิ่มจำนวนแถว)

axis=1

ต่อแนวนอน

เอาคอลัมน์มาปะข้างๆ กันตาม Index (เพิ่มคอลัมน์)

```
pd.concat(objs, axis=0, join='outer', ignore_index=False, keys=None)
```

2.10 ต่อตาราง (CONCAT)

ตัวอย่าง: ต่อแนวตั้ง (axis=0)

รวมข้อมูลสาขากรุงเทพฯ (df_bkk) และสาขาเชียงใหม่ (df_cm)

```
pd.concat([df_bkk, df_cm], axis=0, ignore_index=True)
```

Index	Name	Age	City
0	Alice	25	Bangkok
1	Bob	30	Bangkok
2	Charlie	35	Chiang Mai
3	David	28	Chiang Mai

2.10 ต่อตาราง (CONCAT)

ตัวอย่าง: ต่อแนวนอน (axis= 1)

แปะคอลัมน์คะแนนสอบ (df_score) ไว้ข้างๆ ข้อมูลสาขาที่ 1 (df_bkk)

```
pd.concat([df_bkk, df_score], axis=1)
```

Name	Age	City	Score	Grade
Alice	25	Bangkok	85	A
Bob	30	Bangkok	92	A+

2.9–2.10 สรุป

ข้อแตกต่าง `pd.concat()` vs `pd.merge()`

`pd.concat()`

วางตารางต่อกันตรง ๆ

ต่อล่างหรือแปะข้าง อิงตาม Index / Axis เป็นหลัก

`pd.merge()`

Join ตามคีย์ (**Key-based Join**)

คล้าย SQL JOIN อิงตามค่าในคอลัมน์ร่วมกัน

SESSION 3

Data visualization

สร้างกราฟจากข้อมูลตาราง ด้วย Seaborn

3.1 รู้จัก SEABORN

Seaborn คืออะไร?

ไลบรารีวาดกราฟเชิงสถิติ สร้างต่อยอดจาก Matplotlib รับ DataFrame ของ Pandas ได้โดยตรง

Matplotlib

พื้นฐานของการวาดกราฟ

ปรับแต่งได้ทุกจุด แต่โค้ดยาว ใช้คู่กับ Seaborn
ตอนใส่ชื่อกราฟ / แกน

Seaborn

กราฟสถิติในบรรทัดเดียว

ใส่ชื่อคอลัมน์ได้เลย แยกสีตามกลุ่มด้วย hue สวย
ตั้งแต่ค่าเริ่มต้น

Plotly

กราฟโต้ตอบได้

ชี้เมาส์ดูตัวเลข ชูมได้ เหมาะกับการสำรวจข้อมูล
และ dashboard

Workshop นี้: เน้น Seaborn สำหรับกราฟในรายงานและสไลด์

3.1 รู้จัก SEABORN

เตรียมเครื่องมือ (**import**) และสีประจำกลุ่ม

```
import seaborn as sns          # โหลดกราฟิก ตั้งชื่อย่อว่า sns
import matplotlib.pyplot as plt # ใช้ชื่อกราฟ / แกน และแสดงผล
sns.set_theme(style="whitegrid") # พื้นหลังสีขาว มีเส้นตาราง

# กำหนดสีประจำกลุ่มไว้ครั้งเดียว แล้วใช้ซ้ำทุกกราฟ
SEX_COLORS = {"male": "#2a78d6", "female": "#eb6834"}
ALIVE_COLORS = {"no": "#9e9d98", "yes": "#1baf7a"}
```

ใช้สีชุดเดิมทุกกราฟ: ผู้ชายเป็นสีน้ำเงินเสมอ ผู้อ่านไม่ต้องเดาความหมายของสีใหม่ทุกรูป

3.1 รู้จัก SEABORN

รูปแบบคำสั่ง 4 ขั้นตอน ใช้ซ้ำได้ทุกกราฟ

```
plt.figure(figsize=(7, 4))      # 1. สร้างพื้นที่วาด (กว้าง, สูง หน่วยนิ้ว)
sns.histplot(data=df_clean, x="age") # 2. วาดกราฟ: ตาราง + คอลัมน์
plt.title("Age distribution")    # 3. ใส่ชื่อกราฟ
plt.xlabel("Age (years)")        #   และชื่อแกน
plt.show()                      # 4. แสดงผล
```

ข้อควรระวัง: ชื่อกราฟและแกนใน Matplotlib / Seaborn ควรเป็นภาษาอังกฤษ เพราะภาษาไทยมักแสดงเป็นสีเหลี่ยม (□□□)

3.1 รู้จัก SEABORN

เลือกกราฟให้ตรงกับคำถาม

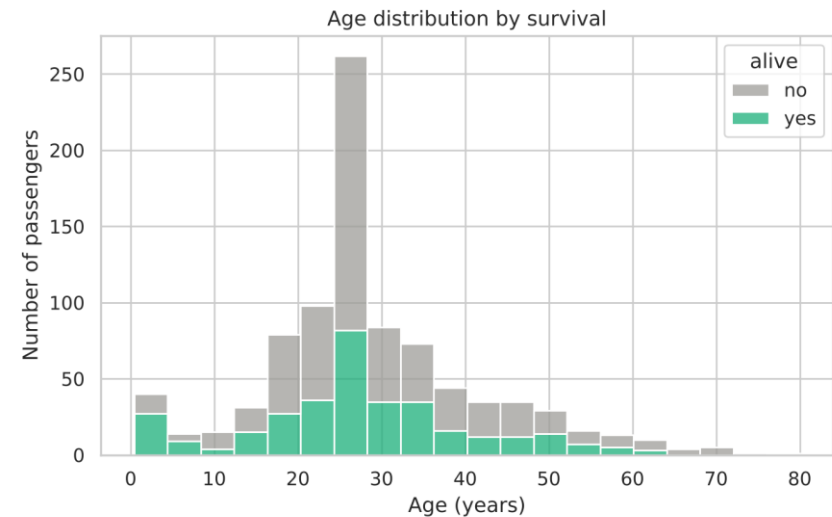
กราฟ	ใช้ตอบคำถามแบบไหน	คำสั่ง
Histogram	ตัวเลขกระจายตัวอย่างไร?	sns.histplot()
Bar chart	กลุ่มไหนมาก / น้อยกว่ากัน?	sns.barplot()
Line chart	ค่าเปลี่ยนไปอย่างไรตามลำดับ (เวลา, อายุ)?	sns.lineplot()
Heatmap	ตาราง 2 มิติ ช่องไหนสูง / ต่ำ?	sns.heatmap()
Scatter plot	ตัวเลข 2 คอลัมน์สัมพันธ์กันไหม?	sns.scatterplot()

เคล็ดลับ: สรุปรูปข้อมูลด้วย `groupby` ก่อน แล้วค่อยวาด จะรู้เสมอว่าตัวเลขในกราฟมาจากไหน

3.2 HISTOGRAM

Histogram: ข้อมูลกระจายตัวอย่างไร?

```
plt.figure(figsize=(7, 4))
sns.histplot(data=df_clean, x="age",
             hue="alive", bins=20,
             multiple="stack",
             palette=ALIVE_COLORS)
plt.title("Age distribution by survival")
plt.xlabel("Age (years)")
plt.show()
```



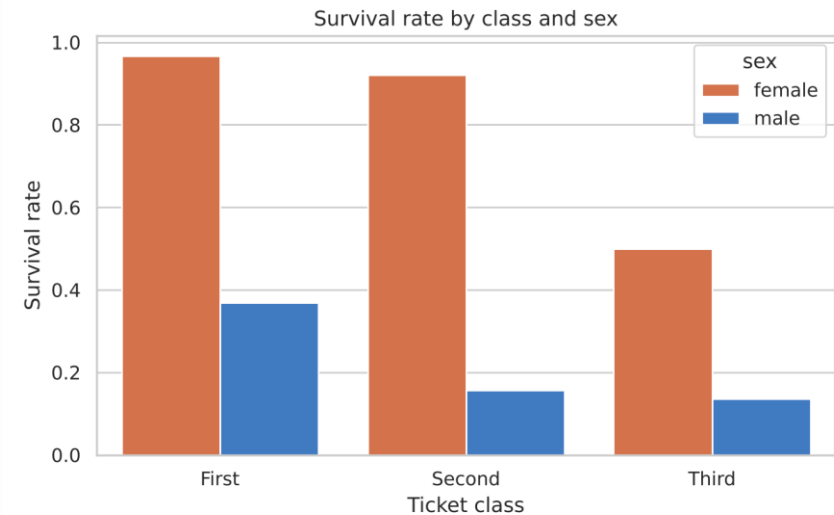
bins = จำนวนแท่ง • **hue** = แยกสีตามกลุ่ม • แท่งสูงผิดปกติที่อายุ 28 ปี มาจากการเติมค่า median

3.3 BAR CHART

Bar chart: กลุ่มไหนมากกว่ากัน?

```
rate = (df_clean
        .groupby(["class", "sex"])["survived"]
        .mean()
        .reset_index(name="survival_rate"))

sns.barplot(data=rate, x="class",
            y="survival_rate", hue="sex",
            palette=SEX_COLORS)
plt.show()
```

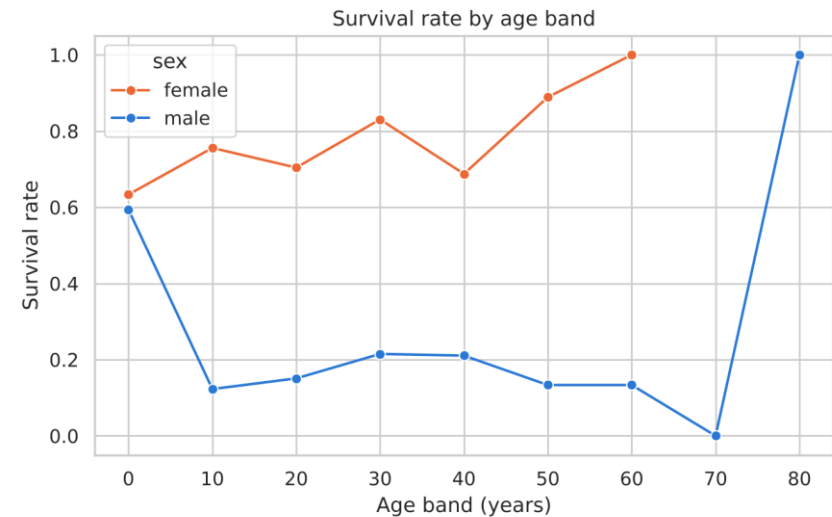


สรุปด้วย **groupby** ก่อนวาด · ถ้าแค่อยากรับจำนวน ใช้ `sns.countplot(data=df_clean, x="class")`

3.4 LINE CHART

Line chart: ค่าเปลี่ยนไปอย่างไรตามลำดับ?

```
df_clean["age_decade"] = (  
    df_clean["age"] // 10 * 10).astype(int)  
trend = (df_clean  
    .groupby(["age_decade", "sex"])  
    .agg(survival_rate=("survived", "mean"),  
        n=("survived", "size"))  
    .reset_index())  
sns.lineplot(data=trend, x="age_decade",  
             y="survival_rate", hue="sex",  
             marker="o", palette=SEX_COLORS)
```

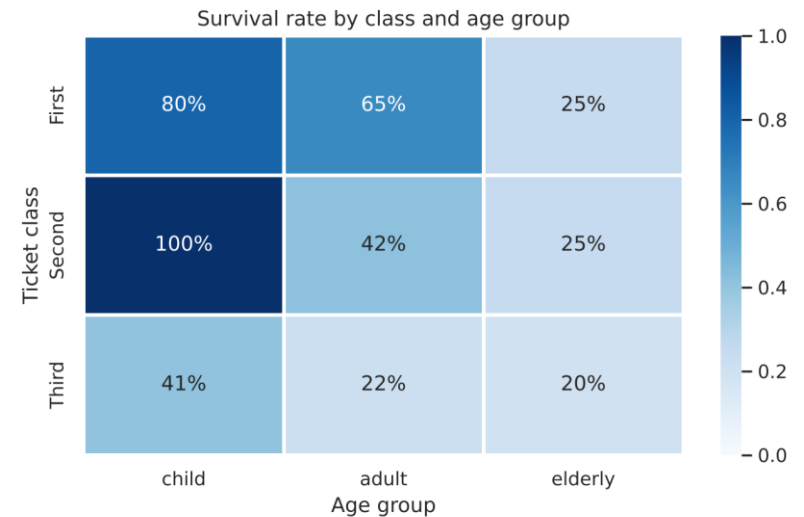


ระวังกลุ่มที่มีคนน้อย: จุดผู้ชายอายุ 80 ปีที่พุ่งไป 100% มาจากคนเพียง 1 คน ดูจำนวน n คู่กับอัตราเสมอ

3.5 HEATMAP

Heatmap: ดูตาราง 2 มิติด้วยสี

```
table = (df_clean
         .groupby(["class", "age_group"])
         ["survived"].mean()
         .unstack())
table = table[["child", "adult", "elderly"]]
sns.heatmap(table, annot=True, fmt=".0%",
            cmap="Blues", vmin=0, vmax=1,
            linewidths=2, linecolor="white")
plt.show()
```

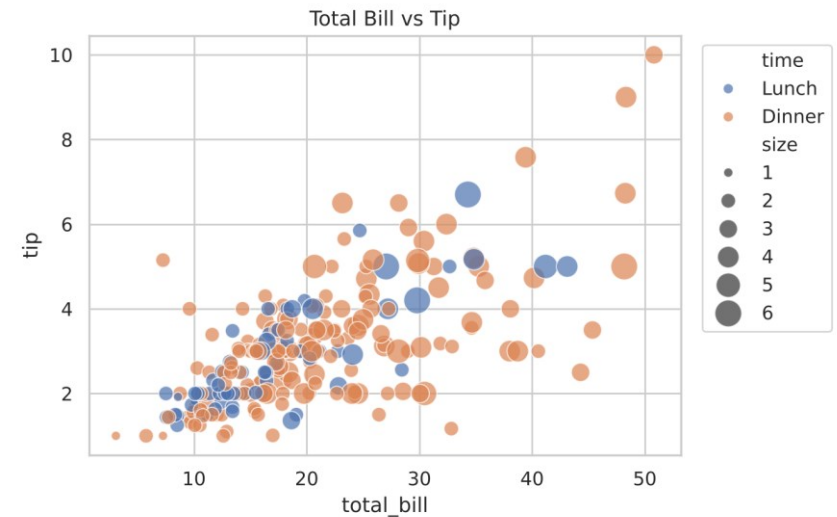


annot = แสดงตัวเลขในช่อง · **fmt=".0%"** = แสดงเป็น % · **cmap="Blues"** = สีเดียว อ่อน → เข้ม

3.6 SCATTER PLOT

Scatter plot: 2 ตัวเลขสัมพันธ์กันไหม?

```
sns.scatterplot(  
    data=tips,  
    x="total_bill", y="tip",  
    hue="time", size="size", sizes=(30, 250),  
    alpha=0.7,  
)  
plt.title("Total Bill vs Tip")  
plt.legend(bbox_to_anchor=(1.02, 1),  
           loc="upper left")  
plt.show()
```



1 จุด = 1 แกว · จุดเรียงขึ้นไปทางขวา แปลว่าบิลยิ่งแพง ทิปยิ่งมาก (ใช้ข้อมูล tips ร้านอาหาร 244 บิล)

3.6 SCATTER PLOT

พารามิเตอร์ของ scatterplot

พารามิเตอร์	ทำอะไร
<code>x="total_bill", y="tip"</code>	แกนนอน = ยอดบิล, แกนตั้ง = ทิป
<code>hue="time"</code>	สีของจุด แยกตามมื้อ (Lunch / Dinner)
<code>size="size"</code>	ขนาดของจุด ตามจำนวนคนในโต๊ะ
<code>sizes=(30, 250)</code>	ขนาดจุดเล็กสุด / ใหญ่สุด ให้เห็นความต่างชัดเจน
<code>alpha=0.7</code>	ความโปร่งใส (0 = ใส, 1 = ทึบ) เห็นจุดที่ซ้อนกัน
<code>plt.legend(bbox_to_anchor=(1.02, 1), ...)</code>	ย้าย legend ออกไปนอกกราฟด้านขวา ไม่ให้บังจุด

อย่าใส่มิติเยอะเกินไป: ถ้ากราฟเริ่มอ่านยาก ให้ตัด size ออกก่อน เพราะคนแยกขนาดจุดได้ยากกว่าแยกสี

3.7 สรุป

พารามิเตอร์ที่ใช้บ่อยใน Seaborn

พารามิเตอร์	ทำอะไร	ใช้กับ
hue=	แยกสีตามกลุ่ม	hist, bar, line, scatter
palette=	กำหนดสีเอง เช่น SEX_COLORS	hist, bar, line, scatter
bins=	จำนวนแท่ง	histplot
order=	เรียงลำดับกลุ่มบนแกน	barplot
marker="o"	ใส่จุดบนเส้น	lineplot
annot=, fmt=, cmap=	ตัวเลขในช่อง, รูปแบบตัวเลข, ชุดสี	heatmap
alpha=	ความโปร่งใสของจุด	scatterplot

ดูพารามิเตอร์ทั้งหมดของคำสั่ง: พิมพ์ `help(sns.histplot)` แล้วรัน หรือเอาเมาส์ชี้ที่ชื่อคำสั่งใน Colab